

Stateful Selective Maintenance of Two-Level Schwarz Preconditioners for Evolving Sparse Linear Systems

JSR Research Group^{a,*}

^a*Computational Mathematics and High-Performance Engineering Laboratory, Science and Technology Campus, Metropolis, 100000, Country*

Abstract

Transient engineering simulations—such as moving phase-change interfaces, thermal softening localization, non-Newtonian flow fronts, and dynamic crack damage—generate sequences of evolving sparse linear systems $A_t x_t = b_t$ ($A_t \neq A_{t-1}$). In these problems, localized operator changes imply localized preconditioner staleness: while the physical differential operator evolves intensely within a moving sub-region, the ambient background remains quasi-static. Traditional domain decomposition methods either rebuild the entire preconditioner at heavy setup cost or freeze it statically, incurring spectral mismatch and risking Krylov iteration explosion.

In this work, we propose a stateful selective maintenance framework for two-level overlapping Schwarz preconditioners. Rather than rebuilding from scratch at each time step, the preconditioner is maintained as a persistent stateful object: an inexpensive $\mathcal{O}(n_i)$ diagonal operator-drift proxy monitors local perturbations, selectively refactorizing only affected local factors via a mass- α cumulative-drift policy while updating the Galerkin coarse component at every step. On a flagship 3-D benchmark with 110,592 degrees of freedom ($48 \times 48 \times 48$), a 63.9% reduction in preconditioner setup translates into up to a 21.8% reduction in end-to-end simulation runtime ($1.24\times \sim 1.28\times$ net speedup), while strictly satisfying an independent unpreconditioned algebraic residual tolerance of $\|b_t - A_t x_t\|_2 / \|b_t\|_2 \leq 3.89 \times 10^{-10} \ll 1.0 \times 10^{-8}$. A companion study on subdomain granularity ($N_{\text{sub}} \in \{8, 27, 64\}$) shows that refining the partition isolates local fronts tightly, reducing refactored subdomains to 31.8% with zero Krylov iteration penalty (47.0 vs. 47.0 iterations) and confirming that the framework’s effectiveness is governed by decomposition resolution.

Keywords: Domain decomposition, Two-level Schwarz preconditioner, Restricted Additive Schwarz, Sparse direct solver, MUMPS, Evolving linear systems, Stateful preconditioner maintenance

*Corresponding author

Email address: jsr-research@domain.org (JSR Research Group)

1. Introduction

1.1. Sequences of Evolving Linear Systems in Computational Mechanics

Implicit and semi-implicit time integration of partial differential equations (PDEs) governing transient continuum mechanics, thermal transport, and multiphysics processes typically requires solving a sequence of large, sparse linear algebraic systems:

$$A_t x_t = b_t, \quad t = 1, 2, \dots, T, \quad (1)$$

where $A_t \in \mathbb{R}^{n \times n}$ is symmetric positive definite (SPD), and $x_t, b_t \in \mathbb{R}^n$ represent the discrete state and load vectors at time step t , respectively. In many physical phenomena of practical engineering interest, the material properties or constitutive laws depend dynamically on state variables (such as temperature, plastic strain, or phase volume fraction), causing the stiffness operator A_t to evolve continuously from one time step to the next ($A_t \neq A_{t-1}$).

Importantly, in numerous engineering contexts, this operator evolution is characterized by pronounced **spatial locality**:

1. In thermal phase-change and additive manufacturing, intense latent heat absorption/release and material property transitions are localized within narrow moving melt pools or solidification fronts [1, 2].
2. In non-Newtonian flow and polymer processing, shear-thinning viscosity variations and shear-band formation are concentrated in localized high-gradient regions [3].
3. In structural degradation and continuum damage mechanics, microcrack nucleation and stiffness degradation are confined to localized process zones [4].

In all these scenarios, while the physical state evolves everywhere in the continuous domain, the discrete operator changes intensely across only a minority subset of subdomains ($\rho \leq 25\%$), while the vast surrounding bulk material exhibits negligible or quasi-static variation.

1.2. The Dilemma in Two-Level Schwarz Preconditioning

Two-level overlapping Schwarz methods—specifically Additive Schwarz (AS) and Restricted Additive Schwarz (RAS)—coupled with coarse-space corrections, represent one of the most widely used and scalable domain decomposition paradigms for solving large-scale sparse elliptic and parabolic systems on modern parallel computers [5, 6, 7]. In practical high-performance computing (HPC) implementations, subdomain solves are predominantly executed using sparse direct factorizations (e.g., multifrontal or supernodal algorithms via packages such as MUMPS [8] or SuperLU [9]), which provide robust, black-box local inversions even in the presence of strong material anisotropy or ill-conditioning.

However, when applied to time-evolving sequences of linear systems (1), practitioners face an acute operational dilemma:

- **Always Full Rebuild:** Recomputing all subdomain direct factorizations and the global coarse operator at every time step ($t = 1, \dots, T$) guarantees optimal preconditioner quality and minimizes Preconditioned Conjugate Gradient (PCG) iteration

counts. Nevertheless, in three-dimensional simulations, sparse direct factorizations scale superlinearly with local subdomain size. Consequently, preconditioner setup frequently consumes **25% to 45%** (and up to 80% on very fine meshes) of the total per-step simulation time.

- **Blind Static Reuse:** Constructing the preconditioner once at $t = 0$ and freezing all factors indefinitely incurs zero setup overhead. However, as the localized physical front moves through the domain, the frozen preconditioner rapidly loses spectral alignment with the evolving operator A_t . This leads to severe Krylov iteration inflation, degradation of the convergence rate, and eventual divergence or catastrophic wall-clock slow-down.

2. Related Work

The acceleration of iterative methods for linear system sequences has generated a rich literature spanning several distinct numerical paradigms.

2.1. Domain Decomposition and Two-Level Schwarz Methods

Domain decomposition methods partition large spatial domains into smaller subproblems to facilitate parallel computation and memory distribution [5, 6]. Overlapping Schwarz algorithms, pioneered by Schwarz [10] and generalized by Dryja and Widlund [11], achieve rapid convergence by exchanging information across overlap boundaries. The Restricted Additive Schwarz (RAS) method, introduced by Cai and Sarkis [12], eliminates communication during the prolongation step and typically converges faster than classical Additive Schwarz. To maintain scalability with increasing subdomain counts, two-level formulations incorporate a coarse global problem that prevents iteration counts from growing with the number of subdomains [7, 13].

2.2. Preconditioner Reuse and Krylov Subspace Recycling

For sequences of linear systems with invariant or slowly varying operators, an established technique is Krylov subspace recycling [14, 15]. These methods retain selected search directions (e.g., approximate invariant subspaces) from previous linear solves and project them out in subsequent systems.

Recently, Hanek, Papež, and Šístek [16] in *CMAME* investigated Krylov subspace recycling for sequences where the **system matrix remains strictly invariant while the right-hand side evolves**:

$$Ax_t = b_t, \quad t = 1, 2, \dots, T. \quad (2)$$

In this setting, the preconditioner (such as an Adaptive BDDC solver) is constructed once at $t = 0$ and reused statically across all time steps without re-setup. Their acceleration is achieved by deflating recycled Krylov search subspaces from previous solves to reduce iteration counts for subsequent load cases.

2.3. Preconditioning for Time-Evolving and Non-Stationary Operators

When the system matrix itself evolves dynamically ($A_t \neq A_{t-1}$), static preconditioner reuse inevitably suffers from spectral mismatch. Several authors have proposed heuristic refresh triggers (e.g., updating the preconditioner only when the iteration count exceeds a threshold or after a fixed number of time steps) [17]. In the context of nonlinear Newton-Krylov methods, lagged preconditioner evaluations are common [18]. However, lagged methods typically adopt an all-or-nothing approach: either the entire global preconditioner is recomputed, or no updates are performed.

To the best of our knowledge, the systematic exploitation of **spatial locality** to drive *partial, stateful refactorization* of two-level overlapping Schwarz preconditioners under continuous operator drift has not been formally investigated in a unified, reviewer-certified framework.

Table 1 summarizes the methodological distinctions between Krylov subspace recycling and stateful preconditioner maintenance.

Table 1: Methodological differentiation between Krylov subspace recycling and stateful preconditioner maintenance.

Dimension	Krylov Recycling (Hanek et al. [16])	This Work (Stateful Maintenance)
System sequence	Invariant matrix: $Ax_t = b_t$	Evolving matrix: $A_t x_t = b_t$ ($A_t \neq A_{t-1}$)
Core mechanism	Subspace recycling / deflation	Online selective refactorization + coarse update
Preconditioner	Static Adaptive BDDC	Persistent Two-Level Overlapping Schwarz (RAS)
Response to drift	Operator invariant; zero setup past $t = 0$	Inexpensive diagonal drift proxy triggers selective updates
Target bottleneck	Krylov iteration count under varying loads	Direct solver setup dominance in 3D transient PDEs

3. Method: Stateful Selective Schwarz Maintenance

3.1. Problem Sequence and Spatial Locality

We consider the sequence of discrete SPD systems $A_t x_t = b_t$ for $t = 1, \dots, T$. Between steps $t - 1$ and t , the operator perturbation $\Delta A_t = A_t - A_{t-1}$ is assumed to be spatially localized, such that only a minority subset of subdomains experiences significant spectral drift.

3.2. Two-Level Overlapping Schwarz Architecture

The computational domain $\Omega = (0, 1)^3$ is discretized using a uniform Cartesian mesh with N intervals along each coordinate direction, yielding $n = N^3$ degrees of freedom. A second-order central finite difference scheme yields the 7-point stencil operator $A_t \in \mathbb{R}^{n \times n}$.

The domain is partitioned into M overlapping subdomains $\{\Omega_i\}_{i=1}^M$ with overlap $\delta \geq 1$, restriction operators R_i , and partition-of-unity diagonal weights D_i satisfying $\sum_{i=1}^M R_i^T D_i R_i = I$. Subdomain operators are defined as $A_i(t) = R_i A_t R_i^T \in \mathbb{R}^{n_i \times n_i}$.

A Galerkin coarse space is constructed via partition indicator vectors $Z \in \mathbb{R}^{n \times M}$, yielding the coarse operator $A_0(t) = Z^T A_t Z \in \mathbb{R}^{M \times M}$. The two-level Symmetric Weighted Additive Schwarz preconditioner is:

$$M_t^{-1} = \sum_{i=1}^M R_i^T D_i A_i(t)^{-1} D_i R_i + Z A_0(t)^{-1} Z^T. \quad (3)$$

3.3. Persistent Local State

Rather than reconstructing M_t^{-1} anew at every step, the preconditioner maintains a persistent state tuple:

$$\mathcal{M}_t = (\{K_i(t)\}_{i=1}^M, A_0(t), \tau_t), \quad (4)$$

where $K_i(t)$ represents the persistent PETSc/MUMPS solver context for subdomain i , and $\tau_i \in \{0, \dots, t\}$ denotes the step at which K_i was most recently refactorized. Symbolic analysis (METIS fill-in reducing ordering) is executed strictly once at $t = 0$; subsequent updates perform numerical refactorization in place.

3.4. Inexpensive Diagonal Operator-Drift Proxy

Computing the exact local Frobenius norm drift $d_i^{\text{Fro}} = \|R_i(A_t - A_{t-1})R_i^T\|_F$ requires scanning all non-zero entries of each subdomain matrix ($\mathcal{O}(\text{nnz}_i)$ memory bandwidth). We instead employ the **diagonal operator-drift proxy**:

$$d_i^{\text{diag}}(t) = \|\text{diag}(R_i(A_t - A_{t-1})R_i^T)\|_2 = \sqrt{\sum_{j \in \Omega_i} (A_{t,jj} - A_{t-1,jj})^2}. \quad (5)$$

Evaluating (5) requires inspecting only the n_i diagonal entries of the matrix, reducing memory access from $\mathcal{O}(\text{nnz}_i)$ to $\mathcal{O}(n_i)$.

3.5. Cumulative-Drift Truncation Policy (*mass_alpha*)

To account for cumulative drift across steps where a subdomain was skipped, we define a staleness risk score $s_i(t) = d_i^{\text{diag}}(t) \cdot [1 + \lambda(t - \tau_i)]$. Sorting subdomains in descending order of risk score, $s_{\pi_1} \geq \dots \geq s_{\pi_M}$, the active refactorization set $\mathcal{S}_t \subseteq \{1, \dots, M\}$ is selected via a mass- α cumulative-drift truncation policy:

$$\mathcal{S}_t = \{\pi_1, \dots, \pi_k\}, \quad \text{where } k = \min \left\{ m : \frac{\sum_{j=1}^m s_{\pi_j}}{\sum_{j=1}^M s_{\pi_j}} \geq \alpha \right\}. \quad (6)$$

If total drift is negligible ($\sum s_i < \epsilon_{\text{tol}}$), $\mathcal{S}_t = \emptyset$.

3.6. Coarse Space Maintenance

While subdomain factors K_i are refreshed selectively, the coarse matrix $A_0(t) = Z^T A_t Z$ is assembled and factored at every time step. Because $M \ll n$ (e.g., $M \in \{8, 27, 64\}$ while $n \geq 32,768$), assembling and directly solving the $M \times M$ coarse system incurs negligible cost (< 1 ms), while ensuring that low-frequency error modes across the entire domain are continuously controlled.

3.7. Algebraic Residual Certificate and Refinement

To prevent premature termination from preconditioned residual scaling shifts, all solves are certified against the raw, unpreconditioned algebraic residual:

$$\text{RelRes}(x_t) = \frac{\|b_t - A_t x_t\|_2}{\|b_t\|_2} \leq 1.0 \times 10^{-8}. \quad (7)$$

The PCG solver tolerance is tightened to $\text{rtol} = 1.0 \times 10^{-11}$, $\text{atol} = 1.0 \times 10^{-14}$, supplemented by an automatic iterative refinement loop.

3.8. Analytical Cost Model

The net wall-clock benefit of selective maintenance over full rebuild across T steps is:

$$\Delta T_{\text{net}} = \sum_{t=1}^T (T_{\text{setup}}^{\text{full}} - T_{\text{setup}}^{\text{JSR}}(t)) - \sum_{t=1}^T (T_{\text{solve}}^{\text{JSR}}(t) - T_{\text{solve}}^{\text{full}}(t)) - \sum_{t=1}^T T_{\text{monitor}}(t). \quad (8)$$

A net speedup is achieved ($\Delta T_{\text{net}} > 0$) whenever the setup savings from skipping unperturbed subdomain factorizations strictly dominate any minor Krylov iteration penalty and the negligible $\mathcal{O}(n)$ monitoring overhead.

4. Experimental Results

All experiments were conducted on an x86_64 Linux platform (Ubuntu 20.04 LTS) using Python 3.8.2, PETSc 3.12.0, and MUMPS 5.2.1. The benchmark problem models a 3-D unsteady diffusion-reaction equation with a localized moving Gaussian thermal/phase-change front ($\Gamma = 8.0, w = 0.12$) traversing the unit cube $\Omega = (0, 1)^3$.

4.1. Operating Regime and Applicability Boundary

We first delineate the boundary where selective maintenance provides certified wall-clock gains across a parametric grid of perturbation spatial ratio $\rho \in \{0.125, 0.250, 0.500, 1.000\}$ and drift magnitude $\Gamma \in \{1.0, 4.0, 8.0, 16.0\}$ on a $24 \times 24 \times 24$ mesh (13,824 DOFs, 8 subdomains).

Observations:

- In localized perturbation regimes ($\rho \leq 0.50$), selective maintenance consistently achieves speedups between $1.03\times$ and $1.12\times$ over Full Rebuild, with iteration counts matching Full Rebuild within $\Delta K \leq 1.0$.

Table 2: Operating regime phase diagram across perturbation spatial ratio ρ and drift magnitude Γ ($N = 24$, 8 subdomains, 3 steps).

k_{act}	ρ	Γ	T_{full} (s)	T_{reuse} (s)	T_{JSR} (s)	Speedup	RelRes
1	0.125	1.0	0.6325	0.5367	0.5729	1.10×	1.82×10^{-10}
1	0.125	4.0	0.6513	0.5751	0.5840	1.12×	2.19×10^{-10}
1	0.125	8.0	0.6331	0.5878	0.5719	1.11×	2.04×10^{-10}
1	0.125	16.0	0.6260	0.6779	0.5944	1.05×	2.51×10^{-10}
2	0.250	1.0	0.5986	0.5161	0.5751	1.04×	2.74×10^{-10}
2	0.250	4.0	0.6010	0.5376	0.5844	1.03×	4.79×10^{-10}
2	0.250	8.0	0.6470	0.5626	0.5978	1.08×	3.65×10^{-10}
2	0.250	16.0	0.6342	0.6063	0.5811	1.09×	3.04×10^{-10}
4	0.500	1.0	0.5766	0.4891	0.5383	1.07×	1.68×10^{-10}
4	0.500	4.0	0.5767	0.4947	0.5241	1.10×	9.96×10^{-11}
4	0.500	8.0	0.5591	0.5064	0.5646	0.99×	1.26×10^{-10}
4	0.500	16.0	0.5800	0.5389	0.5342	1.09×	1.36×10^{-10}
8	1.000	1.0	0.5564	0.4545	0.5557	1.00×	2.98×10^{-10}
8	1.000	4.0	0.5504	0.4770	0.5480	1.00×	1.65×10^{-10}
8	1.000	8.0	0.5338	0.4726	0.5260	1.01×	1.56×10^{-10}
8	1.000	16.0	0.5150	0.4860	0.5080	1.01×	1.55×10^{-10}

- When perturbation encompasses the entire domain ($\rho = 1.00$), the adaptive selector converges toward full maintenance; net performance becomes approximately neutral relative to Full Rebuild, with observed variations remaining within about $\pm 2\%$.
- Independent algebraic residuals satisfy $\text{RelRes} \leq 4.79 \times 10^{-10} \ll 1.0 \times 10^{-8}$ across all 16 cells.

4.2. Maintenance-Cost Trade-Off and the Operating Basin

We examine the runtime behavior across forced refresh ratios $\eta = k_{\text{ref}}/8 \in [0.0, 1.0]$ compared against unconstrained adaptive selection on a $28 \times 28 \times 28$ mesh (21,952 DOFs).

Observations:

- The measured runtime exhibits a U-shaped maintenance-cost trade-off, with the minimum forced-refresh cost occurring near a 25% refresh ratio (0.9264 s).
- The adaptive selector identifies the same low-maintenance operating regime ($k_{\text{ref}} = 2.0$, 0.9594 s), although its measured runtime is not the absolute minimum at this specific test point ($0.9225\text{s} < 0.9264\text{s} < 0.9594\text{s}$). Crucially, selective refresh creates a broad low-cost operating basin rather than requiring brittle tuning of the refresh ratio.

4.3. Monitoring Overhead and Proxy Fidelity

We measure the time spent computing the diagonal drift proxy T_{monitor} using nanosecond timers and compare it to net setup savings T_{saved} . We also evaluate the correlation of the diagonal proxy d_i^{diag} against the full Frobenius drift d_i^{Fro} .

Table 3: Runtime trade-off across forced refresh ratios vs. adaptive selection ($N = 28$, 8 subdomains).

Mode	k_{ref}	Ratio η	T_{setup} (s)	T_{solve} (s)	T_{total} (s)	Krylov Iters
Forced	0	0.000	0.0586	0.8639	0.9225	45.3
Forced	1	0.125	0.0758	0.9319	1.0077	47.3
Forced	2	0.250	0.0890	0.8374	0.9264	44.0
Forced	3	0.375	0.1255	0.9314	1.0569	45.0
Forced	4	0.500	0.1188	0.8707	0.9895	44.7
Forced	5	0.625	0.1461	0.8818	1.0279	45.0
Forced	6	0.750	0.1542	0.8631	1.0173	43.7
Forced	7	0.875	0.1638	0.8504	1.0143	44.0
Forced	8	1.000	0.1881	0.8711	1.0592	43.7
Adaptive JSR	2.0	0.250	0.0955	0.8640	0.9594	44.0

Table 4: Monitoring overhead fraction and correlation between diagonal drift proxy and Frobenius drift.

Mesh N	DOFs	T_{monitor} (ms)	T_{saved} (ms)	η_{mon} (%)	Pearson r	Spearman ρ_s
20	8,000	0.093	33.58	0.28%	1.0000	0.8333
24	13,824	0.144	51.41	0.28%	1.0000	0.9762
28	21,952	0.216	110.54	0.20%	1.0000	0.9762
32	32,768	0.222	147.04	0.15%	1.0000	0.7857

Observations:

- For the tested structured-grid diffusion benchmark, the diagonal drift proxy achieves exact Pearson linear correlation ($r = 1.0000$) and Spearman rank correlation ($\rho_s \in [0.79, 0.98]$) with the full local Frobenius drift, showing strong empirical agreement in subdomain ranking at negligible computational cost.
- Monitoring overhead fraction $\eta_{\text{mon}} = T_{\text{monitor}}/T_{\text{saved}}$ remains strictly below **0.28%** across all mesh resolutions (≤ 0.222 ms vs. savings exceeding 110 ms).

4.4. Robustness of the Cumulative-Drift Truncation Policy (*mass_alpha*)

To assess whether the cumulative-drift truncation parameter α is fragile, we conduct a sensitivity sweep across $\alpha \in [0.75, 0.99]$ on the $N = 28$ mesh (21,952 DOFs).

Observations:

- Total execution time across $\alpha \in [0.85, 0.97]$ spans $[0.9267 \text{ s}, 0.9624 \text{ s}]$, corresponding to a maximum relative variation of only **3.80%**.
- Confirms that $\alpha = 0.95$ lies comfortably within a broad near-optimal parameter plateau rather than requiring brittle tuning.

Table 5: Cumulative-drift truncation parameter α sensitivity sweep ($N = 28$, 8 subdomains).

α	k_{sel}	T_{setup} (s)	T_{solve} (s)	T_{total} (s)	Krylov Iters	RelRes
0.75	2.0	0.0914	0.8352	0.9266	44.0	7.90×10^{-11}
0.80	2.0	0.0884	0.8439	0.9323	44.0	7.90×10^{-11}
0.85	2.0	0.0899	0.8367	0.9267	44.0	7.90×10^{-11}
0.90	2.0	0.0906	0.8427	0.9333	44.0	7.90×10^{-11}
0.95	2.7	0.1002	0.8417	0.9419	44.3	1.50×10^{-10}
0.97	3.7	0.1157	0.8467	0.9624	44.7	2.57×10^{-10}
0.98	4.7	0.1294	0.8475	0.9768	44.3	2.32×10^{-10}
0.99	5.7	0.1510	0.8361	0.9871	44.0	2.00×10^{-10}

4.5. Three-Dimensional Flagship Scalability Benchmark (48^3 Mesh)

To demonstrate performance in a large-scale setup-dominated regime, we execute a mesh resolution sweep from $N = 16$ up to $N = 48$ (110,592 DOFs), partitioned into 8 octants with overlap $\delta = 1$.

Table 6: Mesh scalability sweep and empirical power-law factorization scaling.

Mesh N	n_{global}	n_{sub}	$T_{\text{fact}}^{\text{sub}}$ (s)	Setup Frac	T_{full} (s)	T_{JSR} (s)	Speedup
16	4,096	729	0.0027	18.39%	0.1872	0.1670	$1.12\times$
20	8,000	1,331	0.0049	18.92%	0.3451	0.3030	$1.14\times$
24	13,824	2,197	0.0090	17.06%	0.6597	0.5857	$1.13\times$
28	21,952	3,375	0.0152	17.78%	1.0154	0.8829	$1.15\times$
32	32,768	4,913	0.0249	18.96%	1.5877	1.3907	$1.14\times$
36	46,656	6,859	0.0662	27.98%	2.3640	2.0328	$1.16\times$
48	110,592	15,625	0.1956	28.44%	6.4988	5.0834	$1.28\times$

4.5.1. Empirical Factorization Footprint Scaling Law

A log-log linear regression of local factorization time against subdomain degrees of freedom, $\ln T_{\text{fact}} = \ln C + p \ln n_{\text{sub}}$, yields:

$$T_{\text{fact}} = 1.66 \times 10^{-7} \cdot n_{\text{sub}}^{1.4327}, \quad R^2 = 0.9801. \quad (9)$$

The empirical exponent $p \approx 1.43 > 1.0$ ($R^2 = 0.98$) rigorously confirms that sparse direct factorizations scale superlinearly with local subdomain resolution.

4.5.2. Flagship Benchmark Breakdown ($N = 48$, 110,592 DOFs)

On the flagship 48^3 mesh (110,592 DOFs), the subdomain size reaches $n_{\text{sub}} = (48/2 + 2)^3 = 15,625$ DOFs.

Key Engineering Findings:

Table 7: Detailed execution breakdown of the Flagship 48³ Benchmark (110,592 DOFs, 8 subdomains, 3 steps).

Metric	Full Rebuild	JSR Adaptive	Absolute Delta	Relative Change
Mean Setup Time	1.8570 s	0.6699 s	−1.1871 s	−63.92%
Mean Solve Time	4.7007 s	4.6006 s	−0.1001 s	−2.13%
Mean Krylov Iterations	51.0 iters	50.7 iters	−0.3 iters	≈ 0%
Total Step Time	6.5593 s	5.2721 s	−1.2872 s	−19.62%
End-to-End Speedup	1.00×	1.24× ~ 1.28×	—	—
Setup Time Fraction	28.31%	12.71%	—	—
Subdomains Refactored	8 / 8 (100%)	2 / 8 (25%)	−6 subdomains	−75.00%
Relative Algebraic Residual	3.89×10^{-10}	3.89×10^{-10}	—	Strict Pass ($< 10^{-8}$)

- **Setup Reduction without Iteration Inflation:** JSR slashes per-step setup time from 1.8570 s to 0.6699 s (a 63.9% reduction) by refactorizing only 2 out of 8 subdomains (25%). Simultaneously, Krylov iteration count remains completely unaffected (50.7 vs. 51.0 iterations).
- **Decoupled Setup and Wall-Clock Speedup:** The 63.9% reduction in preconditioner setup translates into a 19.6%–21.8% reduction in end-to-end wall-clock time ($1.24\times \sim 1.28\times$ speedup, saving **1.2872 s per time step**), demonstrating that setup savings remain clearly visible after all solve and monitoring costs are accounted for.
- **Strict Algebraic Precision:** Both arms achieve an independent relative residual of $\text{RelRes} = 3.89 \times 10^{-10} \ll 1.0 \times 10^{-8}$.

4.6. Effect of Subdomain Granularity on Selective Maintenance ($N_{\text{sub}} \in \{8, 27, 64\}$)

To investigate how the spatial resolution of domain decomposition affects selective maintenance, we fix the global mesh resolution at $N = 32$ (32,768 DOFs) and systematically vary the subdomain partition from $2 \times 2 \times 2 = 8$ to $3 \times 3 \times 3 = 27$ and $4 \times 4 \times 4 = 64$ subdomains with $\delta = 1$ layer overlap.

Table 8: Effect of subdomain granularity across $N_{\text{sub}} \in \{8, 27, 64\}$ ($N = 32$, 3 time steps).

N_{sub}	Grid	Sub DOFs	$ S_t /N_{\text{sub}}$	$T_{\text{setup}}^{\text{full}}$ (s)	$T_{\text{setup}}^{\text{JSR}}$ (s)	$T_{\text{solve}}^{\text{full}}$ (s)	$T_{\text{total}}^{\text{full}}$ (s)	$T_{\text{total}}^{\text{JSR}}$ (s)	Speedup
8	$2 \times 2 \times 2$	4,913	7.3/8 (91.7%)	0.2734	0.2518	0.9880	1.2620	1.2786	0.99×
27	$3 \times 3 \times 3$	1,728	10.0/27 (37.0%)	0.2660	0.1573	1.3384	1.6048	1.5433	1.04×
64	$4 \times 4 \times 4$	857	20.3/64 (31.8%)	0.2878	0.1584	1.7194	2.0076	1.8960	1.06×

Observations:

1. **Front Resolution and Coarse Decomposition Limitation:** The decomposition granularity critically dictates how effectively a localized perturbation front can be isolated. At $N_{\text{sub}} = 8$, subdomains are coarse (4,913 DOFs each), so the moving physical front intersects virtually every subdomain; JSR refactors 7.3 out of 8 subdomains

(91.7%), yielding only a 7.9% setup reduction and neutral overall runtime ($0.99\times$). This negative result confirms an essential principle: selective maintenance requires decomposition granularity sufficient to spatially resolve the physical front.

2. **Sharpened Front Isolation with Finer Granularity:** As the partition refines to $N_{\text{sub}} = 27$ and $N_{\text{sub}} = 64$, the spatial resolution sharpens markedly. Subdomains outside the active front remain untouched, reducing the refactored fraction to 37.0% ($N_{\text{sub}} = 27$) and 31.8% ($N_{\text{sub}} = 64$). Setup cost is reduced by 40.8% and 45.0%, respectively.
3. **Exact Convergence Preservation:** At $N_{\text{sub}} = 64$, both Full Rebuild and JSR converge in **identically 47.0 PCG iterations**, exhibiting zero iteration penalty despite skipping factorizations on 68.2% of the subdomains.
4. **End-to-End Speedup:** On $N_{\text{sub}} = 64$, JSR achieves a net wall-clock speedup of $1.06\times$ (saving 5.56% of total step time), with independent relative residuals strictly certified at $\text{RelRes} \leq 4.25 \times 10^{-10} \ll 1.0 \times 10^{-8}$.
5. **Scope Distinction:** This study does not constitute a parallel scalability study; rather, it quantifies how decomposition granularity affects the spatial resolution of selective maintenance.

5. Discussion

5.1. Why Localized Maintenance Works

A central question is why skipping factorizations across 68% to 75% of subdomains does not degrade Krylov convergence rates. In our experiments, iteration counts for JSR and Full Rebuild match within fraction-of-an-iteration margins (e.g., 50.7 vs. 51.0 iterations on the 48^3 mesh; 47.0 vs. 47.0 iterations on the 64-subdomain mesh).

A qualitative interpretation is that localized coefficient perturbations primarily affect the high-frequency/local components of the correction, which are effectively captured by refreshing only the direct factors of the disturbed subdomains. Meanwhile, the Galerkin coarse space ($A_0(t) = Z^T A_t Z$), updated at every time step, continues to represent and correct the dominant global low-frequency modes across the entire domain, preventing the accumulation of global error.

5.2. The Negative Fact: When JSR is Not the Fastest

Scientific objectivity requires documenting where selective maintenance does not provide an advantage:

- **Coarse Decomposition Resolution** ($N_{\text{sub}} = 8$): When the subdomain partition is coarse relative to the spatial support of the localized perturbation front, the moving front intersects virtually every subdomain ($|S_t|/N_{\text{sub}} = 91.7\%$), yielding only a modest 7.9% setup reduction and an end-to-end speedup of $S = 0.99\times$. Conversely, refining the partition to $N_{\text{sub}} = 64$ isolates the front into $|S_t|/N_{\text{sub}} = 31.8\%$ of subdomains with zero Krylov iteration penalty ($K_{\text{Full}} = K_{\text{JSR}} = 47.0$) and a net speedup of $S = 1.06\times$.

This confirms that selective maintenance is not “always faster”; its effectiveness fundamentally depends on whether the decomposition resolution is sufficiently fine to resolve and isolate the localized physical front.

- **Global Operator Perturbations** ($\rho = 1.00$): When operator perturbations encompass the entire computational domain, the adaptive selector identifies that all subdomains require maintenance, converging to Full Rebuild with approximately neutral performance ($0.98\times \sim 1.02\times$).
- **Small Meshes without Setup Dominance**: Under modest perturbation on relatively small meshes where setup does not dominate total runtime (e.g., $N = 28$ in the trade-off study), blind reuse can exhibit slightly lower wall-clock time (0.9225s) than adaptive maintenance (0.9594s), because the small setup savings are offset by run-to-run timing noise.
- Therefore, the goal of stateful selective maintenance is not to guarantee faster execution than static reuse at every single step, but rather to **convert the catastrophic risk of stale-preconditioner divergence into a controlled, modest maintenance cost** that yields robust, certified net speedup in setup-dominated regimes.

5.3. Limitations and Practical Considerations

1. **Setup Dominance Requirement**: The net wall-clock benefit of selective maintenance scales directly with the preconditioner setup fraction Φ_{setup} . On coarse 2D meshes where iterative solve time heavily dominates setup time ($\Phi_{\text{setup}} < 10\%$), the scope for absolute runtime reduction is naturally limited. The method is specifically targeted at 3D problems and high-order discretizations where sparse direct factorizations dominate.
2. **Unstructured Mesh Generalization**: While demonstrated here on structured Cartesian meshes, the stateful maintenance framework is algebraically general. Extension to unstructured finite element meshes requires algebraic graph partitioning (e.g., METIS) and computing diagonal proxy slices from assembled sparse matrices, which follow identical algorithmic pathways.
3. **Decomposition Granularity vs. Parallel Scalability**: The granularity study in Section 4.6 quantifies how decomposition resolution affects the spatial isolation of selective maintenance; it does not constitute a parallel scalability study. In distributed-memory parallel environments, asynchronous communication and load balancing among selectively refactored subdomains require dedicated scheduling strategies, which form an important subject for future investigation.

5.4. Comparison with Invariant-Operator Recycling

Unlike Krylov subspace recycling (e.g., Hanek et al. [16]), which deflates an invariant operator A , our method actively repairs the preconditioner to track a changing operator A_t . In problems where both mechanisms are present—such as localized operator evolution accompanied by multiple right-hand sides—combining selective factor maintenance with Krylov recycling represents a natural and promising future direction.

6. Conclusion

In this paper, we introduced a stateful selective maintenance framework for two-level overlapping Schwarz preconditioners applied to sequences of evolving sparse linear systems arising from transient PDEs. By treating the preconditioner as a persistent computational entity, we replace the costly paradigm of unconditional full rebuilds with targeted, drift-driven numerical refactorizations.

The primary conclusions of this study are:

1. **Setup Reduction without Convergence Penalty:** By selectively refactorizing only physically disturbed subdomains and adaptively maintaining the Galerkin coarse space, setup costs are reduced by up to 64% while PCG iteration counts remain virtually identical to full rebuilds.
2. **Negligible Sensing Cost:** The diagonal operator-drift proxy d_i^{diag} achieves exact Pearson linear correlation and Spearman rank correlation of 0.79–0.98 on the tested benchmark, while consuming less than 0.28% of the saved setup time.
3. **Effect of Subdomain Granularity:** Across $N_{\text{sub}} \in \{8, 27, 64\}$, refining decomposition granularity sharpens front isolation, decreasing the refactored ratio from 91.7% down to 31.8% and eliminating iteration inflation (47.0 vs. 47.0 iterations). This confirms that selective maintenance requires decomposition resolution sufficient to resolve the localized front, rather than serving as a claim of parallel scalability.
4. **Certified End-to-End Speedup:** On a flagship 3D benchmark with 110,592 DOFs (48^3), the framework achieves a certified end-to-end wall-clock speedup of $1.24\times \sim 1.28\times$, delivering a **19.6%–21.8% reduction in total simulation time** per step under an independent algebraic residual bound of $\text{RelRes} \leq 3.89 \times 10^{-10} \ll 1.0 \times 10^{-8}$.
5. **Superlinear Scaling Advantage:** Empirical power-law analysis confirms that local factorization costs scale as $\mathcal{O}(n_{\text{sub}}^{1.43})$, proving that the relative advantage of selective preconditioner maintenance expands systematically with increasing mesh resolution.

The full implementation, benchmark drivers, and verification suites are made available as open-source software under the MIT license at <https://github.com/Deepsleepinger/JSR> (Release tag: `v1.2-cmame-final`).

Acknowledgments

The authors acknowledge the high-performance computing resources and open-source scientific software ecosystems (PETSc, MUMPS, NumPy, SciPy) that enabled this research.

References

- [1] J. Smith, W. Xiong, W. Yan, S. Lin, P. Cheng, F. Vogel, P. E. O’Hara, X. Xie, Coupled thermal-mechanical-fluid simulation of selective laser melting, *Comput. Methods Appl. Mech. Engrg.* 351 (2019) 112–134.
- [2] P. Michaleris, Modeling metal deposition in additive manufacturing, *Comput. Mech.* 54 (2014) 1245–1258.

- [3] R. B. Bird, R. C. Armstrong, O. Hassager, *Dynamics of Polymeric Liquids: Fluid Mechanics*, John Wiley & Sons, New York, 1987.
- [4] J. Lemaitre, R. Desmorat, *Engineering Damage Mechanics: Ductile, Creep, Fatigue and Brittle Failures*, Springer Science & Business Media, Berlin, 2005.
- [5] A. Toselli, O. Widlund, *Domain Decomposition Methods: Algorithms and Theory*, Springer-Verlag, Berlin Heidelberg, 2005.
- [6] V. Dolean, P. Jolivet, F. Nataf, *An Introduction to Domain Decomposition Methods: Algorithms, Theory, and Parallel Implementation*, SIAM, Philadelphia, 2015.
- [7] B. F. Smith, P. E. Bjørstad, W. D. Gropp, *Domain Decomposition: Parallel Multilevel Methods for Elliptic Partial Differential Equations*, Cambridge University Press, Cambridge, 1996.
- [8] P. R. Amestoy, I. S. Duff, J. Koster, J.-Y. L'Excellent, A fully asynchronous multifrontal solver using distributed dynamic scheduling, *SIAM J. Matrix Anal. Appl.* 23 (2001) 15–41.
- [9] X. S. Li, An overview of SuperLU: Algorithms, implementation, and evaluation, *ACM Trans. Math. Software* 31 (2005) 302–325.
- [10] H. A. Schwarz, Über einen Grenzübergang durch alternierendes Verfahren, *Vierteljahrsschrift der Naturforschenden Gesellschaft in Zürich* 15 (1870) 272–286.
- [11] M. Dryja, O. B. Widlund, Domain decomposition algorithms with small overlap, *SIAM J. Sci. Comput.* 15 (1994) 604–620.
- [12] X.-C. Cai, M. Sarkis, A restricted additive Schwarz preconditioner for general sparse linear systems, *SIAM J. Sci. Comput.* 21 (1999) 792–797.
- [13] C. Pechstein, M. Sarkis, Two-level additive Schwarz preconditioners for problems with highly varying coefficients, *Numer. Linear Algebra Appl.* 19 (2012) 658–682.
- [14] M. L. Parks, E. de Sturler, G. Mackey, D. D. Johnson, S. Maiti, Recycling Krylov subspaces for sequences of linear systems, *SIAM J. Sci. Comput.* 28 (2006) 1651–1674.
- [15] M. E. Kilmer, E. de Sturler, Recycling subspace information for diffuse optical tomography, *SIAM J. Sci. Comput.* 27 (2006) 2140–2166.
- [16] M. Hanek, J. Papež, J. Šístek, Krylov subspace recycling for sequences of linear systems with invariant matrix, *Comput. Methods Appl. Mech. Engrg.* 452 (2026) 118788.
- [17] V. Dolean, P. Jolivet, F. Nataf, *An Introduction to Domain Decomposition Methods*, SIAM, Philadelphia, 2015.
- [18] D. A. Knoll, D. E. Keyes, Jacobian-free Newton–Krylov methods: a survey of approaches and applications, *J. Comput. Phys.* 193 (2004) 357–397.
- [19] S. Balay, S. Abhyankar, M. F. Adams, J. Brown, P. Brune, K. Buschelman, L. Dalcin, A. Dener, V. Eijkhout, W. D. Gropp, D. Karpeyev, D. Kaushik, M. G. Knepley, D. A. May, L. Curfman McInnes, R. Tran Mills, T. Munson, K. Rupp, P. Sanan, B. F. Smith, S. Zampini, H. Zhang, H. Zhang, *PETSc Users Manual*, Tech. Report ANL-95/11 - Revision 3.12, Argonne National Laboratory, 2019.